

The Alberta Buck - The BasketWheel (prototype)

The BuckBasket Closing Its Own Arbitrage

Perry Kundert

2026-09-27

The BuckBasket is the liquidity of its own TOKEN/BUCK pools. Every time a commodity's price moves in the wider market, the basket's pools are left behind, and an arbitrageur who closes the gap takes the difference out of the basket's position. The fees they pay are the basket's compensation – and, by a well-understood result in market-making, not all of it. The BasketWheel lets the basket close those gaps itself: a *consistency cycle* through a commodity's three prices (TOKEN/BUCK, TOKEN/USDC, BUCK/USDC) that needs no capital, changes no supply, and cannot lose money; run by a *work wheel*, a bounded state machine any caller can advance and that pays its callers from what it captures. The harvest goes to the basket's depositors; a side effect keeps the basket's own valuation – the number BUCK_K is steered by – closer to the truth.

This is a prototype. The contracts exist (`src/wheel/`) and run in a simulated economy; none of it is deployed, audited, or settled. This paper sets out the design and its theory, what it intends to capture that the BuckBasket's other embodiments leave on the table, what the simulation says so far, and the questions and risks still open.

Contents

1	Status: a refinement under test	2
2	Where it sits among the BuckBasket's embodiments	2
3	What the basket leaks today	3
4	The consistency cycle	3
5	Who can take which gap	5
6	The work wheel	5
7	Paying the caller	6
8	What it intends to capture	6
9	What the simulation says so far	7
10	Open questions	7
11	Risks	8

12 Conclusion	9
13 References	9

1 Status: a refinement under test

The BasketWheel refines the BuckBasket; it does not replace it. It sits *beside* an existing basket as its own contract, holding two narrow permissions: to credit captured value to the basket's depositors, and to credit BUCK to its treasury. Everything else it does, anyone could do.

Piece	State (2026-09-27)
The theory (this paper)	written; the numbers are the simulation's, and provisional
Python prototype	built; measured on a simulated economy (the none arm, 60 days)
Solidity v1 (src/wheel/)	built; 16 Foundry tests on real Uniswap V3 pools; driven by the simulation, where it matches the prototype
The bursts (demand and supply shocks, attacks, exits)	running: 31 two-year cells, wheel off and on, not yet judged
Deployment, audit	none

Read every figure below as "in a simulated economy, with a thin and noisy cast of arbitrageurs, over a month", not as a forecast.

2 Where it sits among the BuckBasket's embodiments

Three documents describe the BuckBasket, each a layer on the last:

- **The Direct embodiment** ([alberta-buck-ethereum-direct](#)) defines BUCK directly against its commodities. Each constituent has a TOKEN/BUCK Uniswap V3 pool; the basket's value in BUCK – *bvib* – is read from those pools, and the K controller steers *bvib* to 1. A depositor presents a commodity TOKEN, the basket mints the matching BUCK and adds both to the pool as full-range liquidity, and a receipt records the claim.
- **The BuckBasket as a savings facility** ([alberta-buck-ethereum-basket](#)) explains why that liquidity earns: base commodities revert to the mean, a constant-product pool sells what has risen and buys what has fallen, and the world's arbitrageurs – who neither know nor care what a BUCK is – do the trading and pay the pools' fees for the privilege. A director ([alberta-buck-rebalance](#)) and the monetary operations ([alberta-buck-operations](#)) steer the basket's weights and BUCK's supply.
- **The BasketWheel** (this paper) keeps all of that, and changes one thing: it stops leaving the *consistency* of the basket's pools to strangers.

The basket paper's engine is the arbitrageur "working for free". The arbitrageur is not working for free. Closing a gap on the basket's pool is precisely how they are paid – and they are paid by the basket.

3 What the basket leaks today

A liquidity provider whose pool trails an external price loses to whoever brings it back. The loss has a name – *loss-versus-rebalancing*, LVR (Milionis, Moallemi, Roughgarden and Zhang, 2022) – and a size: for a constant-product pool of value V whose asset moves with volatility σ , about

$$\text{LVR} \sim V * \sigma^2 / 8 \quad \text{per unit time}$$

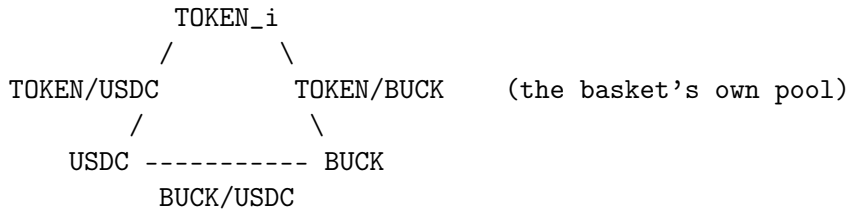
before fees. Fees claw some of it back; how much depends on how often and how far the price moves between trades. The BuckBasket is exactly this provider: it is (nearly) the only liquidity in its TOKEN/BUCK pools, and every move of a commodity’s USD price opens a gap on the basket’s own position.

Two costs follow.

1. **Value.** The arbitrageur who closes the gap keeps the LVR net of the fees. In the simulated economy, whose about \$32M of basket liquidity moves with an inflated 50-67% annual volatility, that is on the order of \$3,000 a day; at a more ordinary 20% volatility, about \$400 a day. (The simulation’s noise overstates it; the order of magnitude is the point.)
2. **Truth.** bvib is a time-average of the TOKEN/BUCK pools. An arbitrageur acts only when a gap pays more than the fees and a margin, so the pools sit anywhere inside that threshold, and bvib – the number the K controller steers – lags the commodities by up to the threshold. In the simulated economy the residual gap at each day’s end has a median of 117-126 basis points and a 90th percentile of 196-257.

4 The consistency cycle

For a constituent i , three pools quote the same commodity:



If the three prices are consistent, going round the triangle returns what went in, less the fees. If they are not, one direction returns more. Going round in that direction, by the right amount, closes the gap and keeps the difference. That is a *consistency cycle*.

4.1 The algebra of a cycle

A constant-product leg with reserves (X, Y) and fee factor g (1 minus the fee) pays, for an input x ,

$$\text{out}(x) = g Y x / (X + g x) = r x / (1 + k x), \quad r = g Y / X, \quad k = g / X$$

– a Moebius map through the origin: r is the leg’s marginal rate at zero size (its price after the fee), k its curvature (the inverse of its depth). Such maps compose into maps of the same form. Feeding one leg into the next,

$$r = r_1 r_2 r_3$$

$$k = k_1 + k_2 r_1 + k_3 r_1 r_2$$

$$\text{cycle}(x) = r x / (1 + k x)$$

so a whole cycle is summarized by two numbers. r is the *gap*: the product of the three prices after fees; the cycle pays exactly when $r > 1$. Its profit, $\text{cycle}(x) - x$, is greatest at

$$x^* = (\sqrt{r} - 1) / k \qquad \text{profit}^* = (\sqrt{r} - 1)^2 / k$$

where the cycle's marginal rate has fallen to 1: the optimal cycle closes the gap down to the fees, no further. A full-range Uniswap V3 position is constant product on its virtual reserves, so the closed form is exact for the basket's own pools and a good approximation elsewhere; where a trade leaves a concentrated range, the realized output falls short of the quote (by about 1.2-1.5% of the value in the simulation).

4.2 What makes a cycle safe

- **No capital, no inventory.** A Uniswap V3 pool pays a swap's output *before* it collects the input, in a callback. The wheel's first leg pays out; the other two legs run inside its callback; what they return pays the first. So a cycle needs nothing on hand, from any starting token – not even a flash loan or a flash mint. USDC exists only between two swaps inside one transaction: the basket never holds a USDC position.
- **No change in supply.** No BUCK is issued or destroyed; value moves from the venues to the basket. A consistency cycle is not a monetary operation, and it never trades BUCK against its basket value (bvib) – that is the undertakings' and the desk's business, under their own rules.
- **It cannot lose money.** The cycle checks its *realized* output against its input and reverts below a minimum edge. An attacker who pushes a pool first only pays the basket to push it back. (What an attacker *can* do is discussed under Risks.)
- **It keeps the gauge true.** Closing the TOKEN/BUCK pools' gaps to the USD venues makes bvib read the external valuation, instead of whatever the last trade left behind – what the external arbitrageurs do today, without their threshold.

4.3 Where the profit lands

The same three swaps, in the same rotation, earn the same whichever token the cycle starts from; the start decides only what the profit is made of.

- **Start from the TOKEN**, and the profit is the commodity. It is added to the basket as *depositor* liquidity, with freshly minted partner BUCK booked as bonus principal – exactly the mechanics the basket's stress fee already uses to pay the remaining depositors – so every receipt's claim grows pro rata, and the basket still burns exactly what it minted.
- **Start from BUCK**, and the profit is BUCK: it funds the callers' reserve (below) up to a cap, and the rest goes to the treasury.

5 Who can take which gap

A searcher going round the triangle pays three pool fees. The basket pays two: on its own TOKEN/BUCK pool it is the only liquidity provider, so the fee it pays comes back to it. With the simulation’s fee tiers (TOKEN/USDC 0.30%, TOKEN/BUCK 0.30%, BUCK/USDC 0.05%):

The gap (roughly)	Who takes it
under 0.35%, plus gas	nobody: a cycle costs more than it earns
0.35% to 0.65%, plus gas	only the basket: the fee it pays on its own pool comes back to it
over 0.65%, plus gas and a margin	searchers, directly – and they pay the basket’s fee on the way
any gap the basket’s own trade opens	the basket, in the same transaction: there is no race

The middle row is the wheel’s natural domain: value no one else can take. The top row it must race for, and on today’s Uniswap V3 pools it cannot stop a searcher from going first. Capturing the large gaps too needs control of the order of swaps – a Uniswap v4 hook that runs the wheel before the first swap of each block, or an auction for the right to go first (the auction-managed AMM of Adams, Moallemi, Reynolds and Robinson, 2024). That is the upgrade path, not the prototype.

6 The work wheel

Someone has to notice a gap and send the transaction. The BasketWheel does not wait for a keeper it pays by the hour: it is a *work wheel*, a bounded state machine that any caller advances with `tick(maxWork, maxScan)`, and that does the block’s work once, however many callers there are.

- **Slots over task kinds.** Each kind of work – a consistency cycle for each constituent, the director’s signal refresh, the treasury’s sweep, the monetary quadrants, the controller’s `compute` – owns a range of slots. A cursor walks them round-robin. Kinds are Solidity mixins over a common chassis, each claiming the slots after its bases’ (the C3 linearization fixes the layout), so a basket’s wheel is assembled from the kinds it wants; a kind with no target configured has no slots.
- **Amortizing the scan, not just the work.** Deciding whether an arbitrage slot is due costs three cold pool reads, about 14,000 gas; scanning six constituents idle would cost more than a piggybacked tick should. So a call examines at most `maxScan` slots, the cursor moves past every slot examined, and the block is marked idle only once consecutive calls have found every slot not due. Many small calls in a block share one scan; after it, each further call is a storage read and a return. Any trade that touches the basket re-arms the wheel within the block.
- **A clock per chain.** The wheel keys its work to the chain’s own block number; where that is not the chain’s own (Arbitrum reports the L1’s), a small adapter supplies the right one.

Folded into transactions a caller sends anyway – the basket’s own deposits and redemptions, an Alberta Buck agent’s routine calls – a tick whose block is done costs a few thousand gas. A tick that works costs what its work costs: about 365,000 gas a cycle, about 630,000-780,000 for a working arbitrage tick with its depositor credit.

7 Paying the caller

The design goal: the calling pays for itself on every chain, without the basket knowing the gas price or the price of ETH.

- **No cure, no pay.** A tick pays only for work done. Callers simulate first and send only when there is work; an idle call is the caller's loss, never the basket's.
- **A share of the value.** A kind that captures value pays its caller a share s of it. A caller calls when $s \times \text{value}$ exceeds its own gas, so the action margin sets itself per chain: on an L1 the wheel acts on large gaps, on a rollup on small ones. s trades how quickly the pools are made consistent against how much of the profit is kept.
- **A reserve for the work that captures nothing.** Upkeep – the director's refresh, the sweep, the controller – earns nothing by itself. A small slice of the basket's yield (and of the arbitrage's value) flows into a reserve; each working tick pays its caller κ of the reserve's balance. So the reserve *builds up* when the wheel is under-called and *pays out less* when it is over-called; at equilibrium a working tick pays about the funding rate divided by the calling rate, and callers call until that equals their gas. The calling rate sets itself, per chain, with no oracle. A cap on the reserve sends the excess to the treasury: the reserve is a gas budget, not a second yield.
- **Competition is harmless.** Callers racing for the same tick bid their priority fees up to the pay less their gas; that transfer is the block builder's, out of the pay. The basket's cost is fixed at the share and κ .

8 What it intends to capture

A basket's excess return over simply holding its commodities comes from three different mispricings, each with its own harvester:

1. **Venue inconsistency** – a TOKEN/BUCK price that disagrees with TOKEN/USDC times BUCK/USDC. Its harvester: **the wheel's consistency cycles**. Its risk: none on a cycle; a side effect on the gauge.
2. **Commodity relative value** – one constituent against another, or against its own trend. Its harvester: the director's rebalancing. Its risk: whipsaw in a secular trend.
3. **BUCK against its basket** – bvib away from 1. Its harvester: the undertakings, the desk, the ladder. Its risk: inventory that persists – a monetary matter, under the monetary rules.

Beneath all three lies the base: the fees on the basket's own pools, less the LVR nobody recaptures. The other embodiments earn the base and leg 2, and leave leg 1 to strangers – who keep it. The wheel's claim is leg 1 alone:

- **LVR, recaptured.** The value a gap represents goes to the basket's depositors (or its treasury), not to the searcher who happens to close it; the fee band's value, which today no one takes, is the basket's by construction.
- **A truer gauge.** bvib stops lagging the commodities behind an arbitrageur's threshold, so the K controller steers on the basket's value rather than on its pools' staleness.

- **Upkeep that pays for itself.** The same wheel, and the same reserve, turn the basket's other maintenance – work that today waits for a keeper, or for a depositor to happen by.

It deliberately does not reach for legs 2 or 3. Relative value is the director's, on its own slow timescale; BUCK against its basket changes the money supply, and belongs to the monetary rules.

9 What the simulation says so far

The simulated economy (`alberta_buck/sim/`) runs the real contracts on an in-process EVM: six commodity TOKENs on their real 2020-2025 price histories, the basket and its director, the K controller, depositors, savers, debtors, and a cast of arbitrageurs. The readings below are its calm arm, days 30-59 (the first month is the pools' own settling).

1. **It pays, on both chains.** The prototype captured about \$1,300-1,700 a day on a \$16-28M basket – 2-3% of its value a year, on this thin and noisy cast – and its calls paid for themselves with room to spare even at 5-gwei L1 gas.
2. **About half of it was the external arbitrageurs'.** Their cycles fell by 30-35%: the wheel took roughly half its value from gaps no one else could take, and half from gaps the arbitrageurs would have closed. (On a real chain searchers would keep more of the large gaps; the simulation orders its actors, not their fees.)
3. **The gauge became truer.** Measured against the basket valued at the commodities' reference prices, `bvib`'s error fell by a third at the median and by 43% at the 90th percentile. `bvib` also moved more – the commodities' real movement reaching it instead of lagging – and more, smaller cycles per call gave most of that added variance back.
4. **The contract matches the prototype.** The Solidity v1, driven by the simulation, captured what the prototype captured (within 10%), and – starting from the TOKEN – credited \$51.8k to the depositors in 30 days, about 3% a year on the savings basket, at about 365,000 gas a cycle through the real identity-bound BUCK.

Two cautions about what these numbers are. The simulation's arbitrageurs demand a 60-basis-point edge on top of the fees, so they leave wide gaps that real searchers, competing toward their gas cost, would not; the wheel's take against a competitive market will be smaller, and more of it will come from the fee band. And one calm month says nothing yet about bursts – demand and supply shocks, a book-loading attack, a liquidity exit – which are being run now, wheel off and on, over two simulated years each.

10 Open questions

1. **The fee band, in practice.** v1 takes only *direct* profit – gaps wider than all three fees. The band only the basket can take needs the basket's own fee to come back within the cycle: a small TOKEN float, or the wheel inside the basket's shell (the savings basket's shell has about 8 KB to spare; the monetary shells about 1-3 KB). That is v2.
2. **The share and kappa.** Fixed, per chain, or adaptive to the reserve's balance and the calling rate?

3. **Cycle sizing.** v1 caps a cycle at a fraction of its *first* leg's reserve, so the two starting tokens size differently; every leg should be capped.
4. **Legs 1 and 2 together.** The wheel re-prices the pools the director reads, and the director's trades open gaps the wheel closes. Do their harvests add, or does one eat the other? (Being measured.)
5. **A competitive market.** How much remains when real searchers compete the large gaps down to gas – and does thinning their participation widen the gaps beyond the wheel's reach?
6. **The large gaps.** A v4 hook that runs the wheel first in each block, or an auction for the right to, would capture what v1 must race for; each has its own trust and liveness questions.
7. **The reserve's funding.** From pool-fee yield, from the arbitrage's value, or both – and which of the upkeep kinds it should pay for.
8. **Depth.** A thin BUCK/USDC pool caps every cycle. The wheel owns no BUCK/USDC position (and should not); how much depth the cycles need, and who provides it, is an open question for any deployment.

11 Risks

- **The callbacks and the credits.** The wheel's swap callback must accept only the pool it is expecting, only inside its own cycle; the cycle entry is callable only by the wheel itself; the basket's credit entry points only by the wheel. v1 does all three and tests them, but these are the surfaces an attacker would probe first, and they have not been audited.
- **A farmable reserve.** Pushing a pool to open a gap costs the attacker more than the share *s* of closing it returns, so the share cannot be farmed. The reserve's kappa, paid per *working* tick, is not tied to the size of the work: when the reserve is large, an attacker might open many small gaps cheaply and collect kappa of the reserve for each. The minimum edge, the cap on the reserve and a pay bounded by the work's value are the defences to test.
- **Displacing the arbitrageurs.** The wheel takes some of what external arbitrageurs earn. If they respond by thinning out, the gaps beyond the wheel's reach – the large ones on v3 – may close more slowly.
- **The wheel's own impact on the gauge.** bvib moves more with the wheel. The evidence so far is that it moves *truer*; a wheel with coarse cycles adds its own lumpiness, which smaller, more frequent cycles remove. K reads bvib, so this matters more than it appears.
- **Liveness.** No calls, no harvest. The reserve pays for calling, and the basket's own traffic can carry ticks, but a wheel nobody turns is simply the BuckBasket without it – no worse, and no better.
- **Gas on an L1.** A working tick is expensive (about 630,000-780,000 gas); on an L1 only large gaps pay, and those are the ones searchers race for. The wheel's case is strongest on rollups.
- **Size.** The monetary basket's shells are close to Ethereum's contract size limit, so the wheel lives beside them as its own contract, holding narrow permissions – a second contract to trust and to upgrade.

- **A trending market.** The wheel does not change the basket’s exposure: in a secular trend a constant-product basket still trails a buy-and-hold of the runaway commodity. The wheel recaptures leakage; it does not cure the basket’s one real risk (the asset that never reverts).

12 Conclusion

The BuckBasket’s promise is that the world’s arbitrageurs keep it honest and pay it to be kept. The first half is true; the second half is only partly true, because the price of keeping a pool honest is paid by the pool. The BasketWheel moves that work in-house: the basket closes its own gaps, with cycles that need no capital and cannot lose, turned by anyone who finds it worth their gas, and it hands the difference to its depositors. If the prototype’s early readings survive the bursts, a competitive market and an audit, the savings basket earns a little more, and – as much to the point – its valuation tells the K controller the truth a little sooner. Until then it is what this paper calls it: a refinement under test.

13 References

- J. Millionis, C. C. Moallemi, T. Roughgarden, A. L. Zhang, *Automated Market Making and Loss-Versus-Rebalancing*, 2022.
- A. Adams, C. C. Moallemi, S. Reynolds, D. Robinson, *am-AMM: An Auction-Managed Automated Market Maker*, 2024.
- Uniswap Labs, *Uniswap v3 Core* (2021): the pay-first swap callback; and Uniswap v4’s hooks.
- The Alberta Buck design notes behind this paper: doc/BASKET-WHEEL.org (the wheel’s design and its measurements), doc/CONVERGENCE.org (the experiments converging the savings basket), src/wheel/ (the contracts) and alberta_buck/sim/ (the simulation).